Research Notes on Reinforcement learning

Reinforcement learning

>> Section 1

In machine learning and optimal control, reinforcement learning (RL) is concerned with how an intelligent agent should take actions in a dynamic environment in order to maximize a reward signal. Reinforcement learning is one of the three basic machine learning paradigms, alongside supervised learning and unsupervised learning. While supervised learning and unsupervised learning algorithms respectively attempt to discover patterns in labeled and unlabeled data, reinforcement learning involves training an agent through interactions with its environment. To learn to maximize rewards from these interactions, the agent makes decisions between trying new actions to learn more about the environment (exploration), or using current knowledge of the environment to take the best action (exploitation). The search for the optimal balance between these two strategies is known as the exploration–exploitation dilemma.

>> Section 2

Generalization and transferability The RL agents trained in specific environments often struggle to generalize their learned policies to new, unseen scenarios. This is the major setback preventing the application of RL to dynamic real-world environments where adaptability is crucial. The challenge is to develop such algorithms that can transfer knowledge across tasks and environments without extensive retraining.

>> Section 3

In natural language processing In recent years Since the early 2020s, reinforcement learning has become, reinforcement learning has become a significant concept in natural language processing (NLP), where tasks are often sequential decision-making rather than static classification. Reinforcement learning is where an agent take actions in an environment to maximize the accumulation of rewards. This framework is best fit for many NLP tasks, including dialogue generation, text summarization, and machine translation, where the quality of the output depends on optimizing long-term or human-centered goals rather than the prediction of single correct label. Early application of RL in NLP emerged in dialogue systems, where conversation was determined as a series of actions optimized for fluency and coherence. These early attempts, including policy gradient and sequence-level training techniques, laid a foundation for the broader application of reinforcement learning to other areas of NLP. A major breakthrough happened with the introduction of reinforcement learning from human feedback (RLHF), a method in which human feedback ratings are used to train a reward model that guides the RL agent. Unlike traditional rule-based or supervised systems, RLHF allows models to align their behavior with human judgments on complex and subjective tasks. This technique was initially used in the development of InstructGPT, an effective language model trained to follow human instructions and later in ChatGPT which incorporates RLHF for improving output responses and ensuring safety. More recently, researchers have explored the use of offline RL in NLP to improve dialogue systems without the need of live human interaction. These methods optimize for user engagement, coherence, and diversity based on past conversation logs and pre-trained reward models. One example is DeepSeek-R1, which incorporates multi-stage training and cold-start data before RL. DeepSeek-R1 achieves performance comparable to OpenAI-o1-1217 on reasoning tasks. This model was trained via large-scale RL without supervised fine-tuning (SFT) as a preliminary step.

>> Section 4

Further reading Annaswamy, Anuradha M. (3 May 2023). "Adaptive Control and Intersections with Reinforcement Learning". Annual Review of Control, Robotics, and Autonomous Systems. 6 (1): 65–93. doi:10.1146/annurev-control-062922-090153. ISSN 2573-5144. S2CID 255702873. Auer, Peter; Jaksch, Thomas; Ortner, Ronald (2010). "Near-optimal regret bounds for reinforcement learning". Journal of Machine Learning Research. 11: 1563–1600. Bertsekas, Dimitri P. (2023) [2019]. Reinforcement Learning and Optimal Control (1st ed.). Athena Scientific. ISBN 978-1-886-52939-7. Busoniu, Lucian; Babuska, Robert; De Schutter, Bart; Ernst, Damien (2010). Reinforcement Learning and Dynamic Programming using Function Approximators. Taylor & Francis CRC Press. ISBN 978-1-4398-2108-4. François-Lavet, Vincent; Henderson, Peter; Islam, Riashat; Bellemare, Marc G.; Pineau, Joelle (2018). "An Introduction to Deep Reinforcement Learning". Foundations and Trends in Machine Learning. 11 (3–4): 219–354. arXiv:1811.12560. Bibcode:2018arXiv181112560F. doi:10.1561/2200000071. S2CID 54434537. Li, Shengbo Eben (2023). Reinforcement Learning for Sequential Decision and Optimal Control (1st ed.). Springer Verlag, Singapore. doi:10.1007/978-981-19-7784-8. ISBN 978-9-811-97783-1. Powell, Warren (2011). Approximate dynamic programming: solving the curses of dimensionality. Wiley-Interscience. Archived from the original on 2016-07-31. Retrieved 2010-09-08. Sutton, Richard S. (1988). "Learning to predict by the method of temporal differences". Machine Learning. 3 (1): 9–44. Bibcode:1988MLear...3....9S. doi:10.1007/BF00115009. Sutton, Richard S.; Barto, Andrew G. (2018) [1998]. Reinforcement Learning: An Introduction (2nd ed.). MIT

Press. ISBN 978-0-262-03924-6. Szita, Istvan; Szepesvari, Csaba (2010). "Model-based Reinforcement Learning with Nearly Tight Exploration Complexity Bounds" (PDF). ICML 2010. Omnipress. pp. 1031–1038. Archived from the original (PDF) on 2010-07-14.

>> Section 5

π : A × S → [0 , 1] π (a , s) = Pr (A t = a ■ S t = s) {\displaystyle {\begin{aligned}&\pi :{\mathcal {A}}\times {\mathcal {S}}\to [0,1]\\&\pi (a,s)=\Pr(A_{t}{=}a\mid S_{t}{=}s)\end{aligned}}}

>> Section 6

Monte Carlo methods Monte Carlo methods are used to solve reinforcement learning problems by averaging sample returns. Unlike methods that require full knowledge of the environment's dynamics, Monte Carlo methods rely solely on actual or simulated experience—sequences of states, actions, and rewards obtained from interaction with an environment. This makes them applicable in situations where the complete dynamics are unknown. Learning from actual experience does not require prior knowledge of the environment and can still lead to optimal behavior. When using simulated experience, only a model capable of generating sample transitions is required, rather than a full specification of transition probabilities, which is necessary for dynamic programming methods. Monte Carlo methods apply to episodic tasks, where experience is divided into episodes that eventually terminate. Policy and value function updates occur only after the completion of an episode, making these methods incremental on an episode-by-episode basis, though not on a step-by-step (online) basis. The term "Monte Carlo" generally refers to any method involving random sampling; however, in this context, it specifically refers to methods that compute averages from complete returns, rather than partial returns. These methods function similarly to the bandit algorithms, in which returns are averaged for each state-action pair. The key difference is that actions taken in one state affect the returns of subsequent states within the same episode, making the problem non-stationary. To address this non-stationarity, Monte Carlo methods use the framework of general policy iteration (GPI). While dynamic programming computes value functions using full knowledge of the Markov decision process, Monte Carlo methods learn these functions through sample returns. The value functions and policies interact similarly to dynamic programming to achieve optimality, first addressing the prediction problem and then extending to policy improvement and control, all based on sampled experience.